

INFORME TÉCNICO

K-POP World



Sistema de Gestión de Agencias de K-Pop

Equipo de Desarrollo Responsable

Equipo #1

Ronald Provance Valladares (C-312) @Ron_1301
Agustín Alberto Carbajal Romero (C-312) @lil_agus03
Daniel Amaranto Mares García (C-312) @amarantomc
Dylan Ramsés Cabrera Morales (C-312) @dylan2108
Sheila Roque Alemán (C-312) @Luminara03

Índice general

1. Introducción	5
1.1. Propósito del Sistema	5
1.2. Alcance	5
2. Descripción del Problema	6
2.1. Problemas Identificados	6
2.1.1. Desorganización de Datos	6
2.1.2. Gestión Compleja de Relaciones	6
2.1.3. Falta de Automatización	6
2.1.4. Seguridad y Control de Acceso	6
3. Solución Propuesta	7
3.1. Objetivo General	7
3.2. Objetivos Específicos	7
4. Arquitectura del Sistema	8
4.1. Modelo de 3 Capas	8
4.1.1. Capa de Presentación	8
4.1.2. Capa de Aplicación	8
4.1.3. Capa de Datos	8
4.2. Componentes Principales	9
4.2.1. Frontend (React)	9
4.2.2. Backend (Node.js + Express)	9
4.2.3. Base de Datos (PostgreSQL)	9
4.3. Patrones de Arquitectura	9
4.3.1. Clean Architecture	9
4.3.2. Dependency Injection	9
5. Diccionario de Datos	11
5.1. Introducción	11
5.2. Tabla: Usuario	11
5.3. Tabla: Agencia	11
5.4. Tabla: Aprendiz	12
5.5. Tabla: Grupo	12
5.6. Tabla: Artista	12
5.7. Tabla: Álbum	13
5.8. Tabla: Canción	13
5.9. Tabla: Actividad	13

5.10. Tabla: Contrato	14
5.11. Tabla: Premio	14
5.12. Tabla: Ingreso	14
5.13. Diagrama Entidad-Relación (MERX)	15
5.14. Propiedades de Datos - Usuario	16
5.15. Propiedades de Datos - Agencia	17
5.16. Propiedades de Datos - Aprendiz	17
5.17. Propiedades de Datos - Grupo	17
6. Especificación Técnica	18
6.1. Stack Tecnológico	18
6.2. Enumeraciones Principales	18
6.2.1. Estado de Usuario	18
6.2.2. Estado de Aprendiz	18
6.2.3. Tipo de Actividad	19
7. Seguridad y Autenticación	20
7.1. Sistema de Autenticación JWT	20
7.2. Control de Acceso por Rol	20
7.2.1. Roles Disponibles en el Sistema	20
7.2.2. Matriz Detallada de Permisos	21
7.2.3. Restricciones Especiales	22
7.3. Matriz de Permisos por Rol	23
7.3.1. AuthMiddleware	23
7.3.2. RoleMiddleware	23
7.4. Protección de Datos Sensibles	23
8. Documentación de APIs	24
8.1. Base URL	24
8.2. Headers Requeridos	24
8.3. Códigos HTTP Utilizados	24
8.4. Endpoints por Módulo	24
8.4.1. Autenticación	24
8.4.2. Usuarios	25
8.4.3. Agencias	25
8.4.4. Grupos	25
8.4.5. Aprendices	25
9. Validaciones y Restricciones	26
9.1. Validaciones por Entidad	26
9.1.1. Usuario	26
9.1.2. Aprendiz	26
9.2. Restricciones de Negocio	27
10. Flujos de Negocio	28
10.1. Flujo 1: Login de Usuario	28
10.2. Flujo 2: Crear Nuevo Grupo	28
10.3. Flujo 3: Evaluar Aprendiz	29

11.Pruebas	30
11.1. Estrategia de Testing	30
11.1.1. Unit Tests	30
11.1.2. Integration Tests	30
11.1.3. E2E Tests	30
11.2. Test Cases Críticos	31
12.Conclusiones	32
12.1. Logros Alcanzados	32
12.2. Beneficios para la Organización	32
12.3. Recomendaciones Futuras	32
A. Anexos	33
A.1. Datos de Prueba	33
A.2. Variables de Entorno Backend	33
A.3. Referencia Rápida de Errores	34
B. Flujos de Negocio	35
B.1. Introducción	35
B.2. Flujo 1: Autenticación de Usuario	35
B.2.1. Descripción	35
B.2.2. Participantes	35
B.2.3. Secuencia del Flujo	35
B.3. Flujo 2: Evaluar Aprendiz (CRÍTICO)	37
B.3.1. Descripción	37
B.3.2. Participantes	37
B.3.3. Secuencia del Flujo	37
B.4. Flujo 3: Crear Nuevo Grupo	39
B.4.1. Descripción	39
B.4.2. Validaciones Críticas	39
B.4.3. Resultado	39
B.5. Flujo 4: Registrar Ingreso	40
B.5.1. Descripción	40
B.5.2. Cálculo de Distribución	40
B.6. Restricciones de Transición de Estado	40
C. Guía de Mantenimiento	41
C.1. Introducción	41
C.2. Estructura de Directorios	41
C.3. Procedimientos de Diagnóstico	42
C.3.1. Verificación de Base de Datos	42
C.3.2. Verificación de Servidor	42
C.4. Procedimientos de Depuración	42
C.4.1. Depuración Node.js	42
C.4.2. Depuración TypeScript/Prisma	43
C.5. Procedimientos de Backup y Restauración	43
C.5.1. Backup de Base de Datos	43
C.5.2. Restauración de Base de Datos	43
C.6. Problemas Comunes y Soluciones	44

C.7. Monitoreo y Performance	44
C.7.1. Índices de Base de Datos	44
C.7.2. Métricas de Servidor	44
C.8. Consideraciones de Seguridad	45
C.8.1. Gestión de Secretos	45
C.8.2. Auditoría y Logs	45
C.8.3. Actualizaciones y Parches	45
D. Diagrama de Clases UML	46
D.1. Introducción	46
D.2. Capas de la Arquitectura	46
D.2.1. Domain Layer (Capa de Dominio)	46
D.2.2. Application Layer (Capa de Aplicación)	47
D.2.3. Infrastructure Layer (Capa de Infraestructura)	48
D.2.4. Presentation Layer (Capa de Presentación)	48
D.3. Patrones de Diseño	49
D.3.1. Dependency Injection	49
D.3.2. Repository Pattern	49
D.4. Relaciones entre Clases	50

Capítulo 1

Introducción

Este documento describe la especificación técnica del **Sistema de Gestión de Agencias de K-Pop**. El sistema es una aplicación web diseñada para facilitar la administración de agencias de música K-Pop, permitiendo la gestión integral de grupos, artistas, aprendices, contratos, actividades y recursos relacionados.

El presente informe establece los lineamientos técnicos, requerimientos, arquitectura, seguridad e implementación del proyecto desarrollado por el Equipo #1 durante el semestre académico 2025-2026.

1.1. Propósito del Sistema

- Centralizar la información de agencias de K-Pop
- Administrar relaciones entre agencias, grupos y artistas
- Gestionar el proceso de evaluación de aprendices
- Mantener registro de contratos y actividades
- Generar reportes y estadísticas

1.2. Alcance

El sistema cubre 14 módulos principales que abarcan todas las operaciones de una agencia de K-Pop, desde la gestión de personal hasta la administración de activos de la empresa.

Capítulo 2

Descripción del Problema

Las agencias de K-Pop enfrentan varios desafíos en su operación diaria:

2.1. Problemas Identificados

2.1.1. Desorganización de Datos

- Información dispersa entre múltiples sistemas
- Dificultad para acceder a datos históricos
- Inconsistencias en registros

2.1.2. Gestión Compleja de Relaciones

- Múltiples relaciones entre entidades (agencias, grupos, artistas)
- Dificil rastrear cambios en status y evaluaciones
- Falta de visibilidad en el progreso de aprendices

2.1.3. Falta de Automatización

- Procesos manuales propensos a errores
- Dificultad en generación de reportes
- Tiempo invertido en tareas administrativas

2.1.4. Seguridad y Control de Acceso

- Información sensible sin protección adecuada
- Falta de auditoría de cambios
- Acceso sin restricciones por rol

Capítulo 3

Solución Propuesta

3.1. Objetivo General

Desarrollar una aplicación web moderna que centralice la gestión de agencias de K-Pop, proporcionando herramientas intuitivas y seguras para la administración de datos y procesos.

3.2. Objetivos Específicos

1. Crear una base de datos relacional normalizada para almacenar información
2. Desarrollar una API REST escalable para acceso a datos
3. Implementar interfaz de usuario intuitiva para cada tipo de usuario
4. Establecer controles de seguridad basados en roles
5. Automatizar procesos críticos del negocio

Capítulo 4

Arquitectura del Sistema

4.1. Modelo de 3 Capas

El sistema implementa una arquitectura de tres capas claramente separadas:

4.1.1. Capa de Presentación

- Tecnología: React + TypeScript + Vite
- Componentes reutilizables
- Context API para estado global
- Interfaz responsiva

4.1.2. Capa de Aplicación

- Tecnología: Node.js + Express + TypeScript
- Controladores HTTP
- DTOs (Data Transfer Objects)
- Lógica de negocio
- Inyección de dependencias

4.1.3. Capa de Datos

- Tecnología: PostgreSQL + Prisma ORM
- Base de datos relacional
- Modelos y esquemas
- Migraciones versionadas

4.2. Componentes Principales

4.2.1. Frontend (React)

- Framework: React 18+ con TypeScript
- Build Tool: Vite
- Estado: Context API
- Estructura: Organizada por feature
- Estilos: CSS modules

4.2.2. Backend (Node.js + Express)

- Runtime: Node.js
- Framework: Express.js
- Lenguaje: TypeScript
- ORM: Prisma
- Patrón: Clean Architecture + Dependency Injection

4.2.3. Base de Datos (PostgreSQL)

- Motor: PostgreSQL 12+
- ORM: Prisma
- Migraciones: Versionadas
- Entidades: 15+ modelos relacionales

4.3. Patrones de Arquitectura

4.3.1. Clean Architecture

La aplicación sigue el patrón Clean Architecture con capas claramente definidas:

Domain Layer (Entidades y Lógica de Negocio)

↓

Application Layer (Casos de Uso y DTOs)

↓

Infraestructure Layer (Repositorios y BD)

↓

Presentation Layer (Controladores)

4.3.2. Dependency Injection

Se implementa un contenedor de inyección de dependencias que permite:

- Registro de servicios centralizados

- Inyección automática en controladores
- Facilidad de testing y mocking

Capítulo 5

Diccionario de Datos

5.1. Introducción

El Diccionario de Datos es una referencia completa de todos los campos, tipos de datos y restricciones de cada tabla en la base de datos. Esta sección es esencial para desarrolladores y administradores de base de datos que realicen mantenimiento del sistema.

5.2. Tabla: Usuario

Cuadro 5.1: Diccionario - Tabla Usuario

Campo	Tipo	Nulo	Restricción	Descripción
id	SERIAL	NO	PK	Identificador único
email	VARCHAR(255)	NO	UNIQUE	Email de login
nombre	VARCHAR(255)	NO		Nombre completo
password	VARCHAR(255)	NO		Contraseña hasheada
rol	ENUM	NO		Rol en el sistema
agencyId	INTEGER	NO	FK	Referencia a Agencia
createdAt	TIMESTAMP	NO	DEFAULT NOW()	Fecha creación
updatedAt	TIMESTAMP	NO	DEFAULT NOW()	Fecha última actualización

5.3. Tabla: Agencia

Cuadro 5.2: Diccionario - Tabla Agencia

Campo	Tipo	Nulo	Restricción	Descripción
id	SERIAL	NO	PK	Identificador único
name	VARCHAR(255)	NO	UNIQUE	Nombre agencia
address	VARCHAR(500)	SÍ		Dirección física
foundationDate	DATE	SÍ		Fecha de fundación
estado	VARCHAR(50)	NO		Estado (Activa/Inactiva)
createdAt	TIMESTAMP	NO	DEFAULT NOW()	Fecha creación
updatedAt	TIMESTAMP	NO		Fecha última actualización

5.4. Tabla: Aprendiz

Cuadro 5.3: Diccionario - Tabla Aprendiz

Campo	Tipo	Nulo	Restricción	Descripción
id	SERIAL	NO	PK	Identificador único
nombre	VARCHAR(255)	NO		Nombre completo
dateOfBirth	DATE	NO		Fecha nacimiento
agencyId	INTEGER	NO	FK	Referencia a Agencia
estado	ENUM	NO		Estado entrenamiento
createdAt	TIMESTAMP	NO	DEFAULT NOW()	Fecha creación
updatedAt	TIMESTAMP	NO		Fecha última actualización

5.5. Tabla: Grupo

Cuadro 5.4: Diccionario - Tabla Grupo

Campo	Tipo	Nulo	Restricción	Descripción
id	SERIAL	NO	PK	Identificador único
nombre	VARCHAR(255)	NO	UNIQUE	Nombre grupo
debut	DATE	SÍ		Fecha de debut
status	VARCHAR(50)	NO		Estado (Activo/Inactivo)
miembros	INTEGER	SÍ	CHECK (1-20)	Cantidad de miembros
idAgency	INTEGER	NO	FK	Referencia a Agencia
createdAt	TIMESTAMP	NO	DEFAULT NOW()	Fecha creación
updatedAt	TIMESTAMP	NO		Fecha última actualización

5.6. Tabla: Artista

Cuadro 5.5: Diccionario - Tabla Artista

Campo	Tipo	Nulo	Restricción	Descripción
id	SERIAL	NO	PK	Identificador único
apprenticeId	INTEGER	NO	FK	Referencia a Aprendiz
groupId	INTEGER	NO	FK	Referencia a Grupo
nombreArtístico	VARCHAR(255)	NO		Nombre artístico
debutfecha	DATE	SÍ		Fecha debut en grupo
status	VARCHAR(50)	NO		Estado en grupo
createdAt	TIMESTAMP	NO	DEFAULT NOW()	Fecha creación
updatedAt	TIMESTAMP	NO		Fecha última actualización

5.7. Tabla: Álbum

Cuadro 5.6: Diccionario - Tabla Álbum

Campo	Tipo	Nulo	Restricción	Descripción
id	SERIAL	NO	PK	Identificador único
titulo	VARCHAR(255)	NO		Título del álbum
groupId	INTEGER	NO	FK	Referencia a Grupo
fechaLanzamiento	DATE	NO		Fecha de lanzamiento
numCanciones	INTEGER	SÍ		Cantidad de canciones
copiaVendidas	INTEGER	SÍ	DEFAULT 0	Copias vendidas
createdAt	TIMESTAMP	NO	DEFAULT NOW()	Fecha creación
updatedAt	TIMESTAMP	NO		Fecha última actualización

5.8. Tabla: Canción

Cuadro 5.7: Diccionario - Tabla Canción

Campo	Tipo	Nulo	Restricción	Descripción
id	SERIAL	NO	PK	Identificador único
titulo	VARCHAR(255)	NO		Título canción
albumId	INTEGER	NO	FK	Referencia a Álbum
duracion	DECIMAL(5,2)	SÍ		Duración en minutos
reproducciones	INTEGER	SÍ	DEFAULT 0	Cantidad de reproducciones
createdAt	TIMESTAMP	NO	DEFAULT NOW()	Fecha creación
updatedAt	TIMESTAMP	NO		Fecha última actualización

5.9. Tabla: Actividad

Cuadro 5.8: Diccionario - Tabla Actividad

Campo	Tipo	Nulo	Restricción	Descripción
id	SERIAL	NO	PK	Identificador único
tipo	ENUM	NO		Tipo actividad
fecha	DATE	NO		Fecha actividad
lugar	VARCHAR(255)	NO		Ubicación
participantes	TEXT	SÍ		JSON con IDs participantes
estado	VARCHAR(50)	NO		Estado actividad
createdAt	TIMESTAMP	NO	DEFAULT NOW()	Fecha creación
updatedAt	TIMESTAMP	NO		Fecha última actualización

5.10. Tabla: Contrato

Cuadro 5.9: Diccionario - Tabla Contrato

Campo	Tipo	Nulo	Restricción	Descripción
id	SERIAL	NO	PK	Identificador único
tipo	VARCHAR(50)	NO		Tipo contrato
parte1Id	INTEGER	NO	FK	Referencia entidad 1
parte2Id	INTEGER	NO	FK	Referencia entidad 2
fechaInicio	DATE	NO		Fecha inicio
fechaFin	DATE	SÍ	CHECK ($\geq$ inicio)	Fecha fin
condiciones	TEXT	SÍ		Condiciones especiales
estado	VARCHAR(50)	NO		Estado contrato
createdAt	TIMESTAMP	NO	DEFAULT NOW()	Fecha creación
updatedAt	TIMESTAMP	NO		Fecha última actualización

5.11. Tabla: Premio

Cuadro 5.10: Diccionario - Tabla Premio

Campo	Tipo	Nulo	Restricción	Descripción
id	SERIAL	NO	PK	Identificador único
nombre	VARCHAR(255)	NO		Nombre del premio
groupId	INTEGER	SÍ	FK	Referencia Grupo
artistaId	INTEGER	SÍ	FK	Referencia Artista
fecha	DATE	NO		Fecha del premio
categoría	VARCHAR(255)	SÍ		Categoría
createdAt	TIMESTAMP	NO	DEFAULT NOW()	Fecha creación

5.12. Tabla: Ingreso

Cuadro 5.11: Diccionario - Tabla Ingreso

Campo	Tipo	Nulo	Restricción	Descripción
id	SERIAL	NO	PK	Identificador único
tipo	VARCHAR(50)	NO		Tipo ingreso
monto	DECIMAL(10,2)	NO	CHECK (>0)	Cantidad en dinero
fecha	DATE	NO		Fecha ingreso
actividadId	INTEGER	SÍ	FK	Referencia Actividad
distribucion	TEXT	SÍ		JSON distribución
createdAt	TIMESTAMP	NO	DEFAULT NOW()	Fecha creación
updatedAt	TIMESTAMP	NO		Fecha última actualización

El sistema cuenta con 15+ entidades principales relacionadas entre sí:

Cuadro 5.12: Entidades Principales

Entidad	Registros Típicos	Descripción
Usuario	10-50	Administradores, gerentes, artistas
Agencia	3-10	Agencias de K-Pop principales
Grupo	20-100	Grupos musicales bajo agencias
Artista	100-500	Miembros de grupos
Aprendiz	50-300	Nuevos talentos en entrenamiento
Contrato	20-100	Acuerdos entre entidades
Álbum	50-300	Lanzamientos discográficos
Canción	500-2000	Composiciones musicales
Actividad	200-1000	Eventos, conciertos, actividades
Premio	50-200	Reconocimientos ganados

5.13. Diagrama Entidad-Relación (MERX)

Ubicación del archivo:

- Versión 1: 1era entrega BD/MERX K-pop Final (2).drawio
- Versión 2: Seminario Problematica/MERX K-pop Final Actualizado.drawio

El Diagrama Entidad-Relación (MERX - Modelo Entidad-Relación eXtendido) representa la estructura lógica de la base de datos. Los diagramas en formato DrawIO pueden visualizarse en la aplicación Draw.io en línea o en VS Code con la extensión correspondiente.

Relaciones Principales

- Agencia → Grupo (1:N) - Una agencia tiene múltiples grupos
- Agencia → Usuario (1:N) - Una agencia tiene múltiples usuarios gestores
- Agencia → Aprendiz (1:N) - Una agencia entrena múltiples aprendices
- Grupo → Artista (1:N) - Un grupo tiene múltiples artistas
- Grupo → Álbum (1:N) - Un grupo produce múltiples álbumes
- Álbum → Canción (1:N) - Un álbum contiene múltiples canciones
- Grupo → Premio (1:N) - Grupos pueden recibir múltiples premios
- Actividad → Ingreso (1:N) - Una actividad genera múltiples ingresos
- Contrato → Partes (M:N) - Contratos vinculan múltiples entidades

Figura 5.1: Diagrama MERX - Entidades y Relaciones del Sistema K-POP World

5.14. Propiedades de Datos - Usuario

Cuadro 5.13: Tabla Usuario

Campo	Tipo	Restricción
id	SERIAL	PRIMARY KEY
email	VARCHAR(255)	UNIQUE, NOT NULL
nombre	VARCHAR(255)	NOT NULL
password	VARCHAR(255)	NOT NULL
rol	ENUM	ADMIN, STAFF, ARTIST, APPRENTICE, MANAGER, DIRECTOR
agencyId	INTEGER	FOREIGN KEY
createdAt	TIMESTAMP	DEFAULT NOW()
updatedAt	TIMESTAMP	DEFAULT NOW()

5.15. Propiedades de Datos - Agencia

Cuadro 5.14: Tabla Agencia

Campo	Tipo	Restricción
id	SERIAL	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
address	VARCHAR(500)	
foundationDate	DATE	
estado	VARCHAR(50)	
createdAt	TIMESTAMP	DEFAULT NOW()
updatedAt	TIMESTAMP	DEFAULT NOW()

5.16. Propiedades de Datos - Aprendiz

Cuadro 5.15: Tabla Aprendiz

Campo	Tipo	Restricción
id	SERIAL	PRIMARY KEY
nombre	VARCHAR(255)	NOT NULL
dateOfBirth	DATE	NOT NULL
agencyId	INTEGER	NOT NULL, FOREIGN KEY
estado	ENUM	ACTIVE, INACTIVE, GRADUATED
createdAt	TIMESTAMP	DEFAULT NOW()
updatedAt	TIMESTAMP	DEFAULT NOW()

5.17. Propiedades de Datos - Grupo

Cuadro 5.16: Tabla Grupo

Campo	Tipo	Restricción
id	SERIAL	PRIMARY KEY
nombre	VARCHAR(255)	NOT NULL, UNIQUE
debut	DATE	
status	VARCHAR(50)	ACTIVE, INACTIVE
miembros	INTEGER	1-20
idAgency	INTEGER	NOT NULL, FOREIGN KEY
createdAt	TIMESTAMP	DEFAULT NOW()

Capítulo 6

Especificación Técnica

6.1. Stack Tecnológico

Cuadro 6.1: Componentes Tecnológicos

Componente	Tecnología	Versión
Frontend	React	18+
Backend	Node.js	18+
Framework API	Express.js	4+
Lenguaje	TypeScript	5+
ORM	Prisma	5+
Base de Datos	PostgreSQL	12+
Build	Vite	5+
Test	Jest	29+

6.2. Enumeraciones Principales

6.2.1. Estado de Usuario

- ADMIN: Acceso total
- STAFF: Gestión de contenido
- ARTIST: Artista del sistema
- APPRENTICE: Aprendiz
- MANAGER: Gerente de agencia
- DIRECTOR: Director ejecutivo

6.2.2. Estado de Aprendiz

- ACTIVE: En entrenamiento
- INACTIVE: Inactivo temporalmente

- GRADUATED: Completó entrenamiento
- SUSPENDED: Suspendido

6.2.3. Tipo de Actividad

- CONCERT: Concierto público
- REHEARSAL: Ensayo privado
- RECORDING: Sesión de grabación
- EVENT: Evento especial
- PRESENTATION: Presentación

Capítulo 7

Seguridad y Autenticación

7.1. Sistema de Autenticación JWT

El sistema implementa autenticación basada en JWT (JSON Web Tokens) con la siguiente estructura:

Cuadro 7.1: Payload JWT

Campo	Descripción
userId	Identificador del usuario
email	Email del usuario
role	Rol del usuario en el sistema
agencyId	Agencia a la que pertenece
iat	Tiempo de emisión (issued at)
exp	Tiempo de expiración (24 horas)

7.2. Control de Acceso por Rol

7.2.1. Roles Disponibles en el Sistema

El sistema implementa 6 roles principales con niveles de acceso progresivos:

Cuadro 7.2: Descripción de Roles

Rol	Descripción y Responsabilidades
ADMIN	Acceso administrativo completo. Gestión de usuarios, permisos, configuración del sistema.
DIRECTOR	Acceso directivo. Aprobación de cambios, reportes, supervisión de agencia.
MANAGER	Gestor de operaciones. Crea y edita grupos, aprendices, actividades, registra ingresos.
STAFF	Personal administrativo. Lectura de datos, creación de nuevos registros con supervisión.
ARTIST	Artista del sistema. Acceso limitado a su información personal y grupo.
APPRENTICE	Aprendiz en formación. Acceso muy limitado, solo su perfil y materiales de entrenamiento.

7.2.2. Matriz Detallada de Permisos

La siguiente matriz indica los permisos (C=Create, R=Read, U=Update, D=Delete) para cada rol por recurso:

Cuadro 7.3: Matriz Completa de Permisos por Rol

Recurso	ADMIN	STAFF	ARTIST	APPRENTICE	MANAGER	DIRECTOR
<i>Usuarios</i>						
Ver usuarios	CRUD	R	R	R	R	R
Crear usuario	CRUD	R	-	-	R	R
Editar usuario	CRUD	R	R*	R*	R	U
Eliminar usuario	CRUD	-	-	-	-	-
<i>Agencias</i>						
Ver agencias	CRUD	R	R	-	R	R
Crear agencia	CRUD	-	-	-	-	-
Editar agencia	CRUD	-	-	-	U	U
<i>Grupos</i>						
Ver grupos	CRUD	R	R	R	R	R
Crear grupo	CRUD	-	-	-	C	C
Editar grupo	CRUD	U	-	-	U	U
Eliminar grupo	CRUD	-	-	-	-	-
<i>Aprendices</i>						
Ver aprendices	CRUD	R	R	R*	R	R
Crear aprendiz	CRUD	C	-	-	C	C
Editar aprendiz	CRUD	U	-	R*	U	U
Evaluar aprendiz	-	-	-	-	C	U
<i>Artistas</i>						
Ver artistas	CRUD	R	R	-	R	R
Crear artista	CRUD	U	-	-	R	R
Editar artista	CRUD	D	R*	-	R	R
<i>Contratos</i>						
Ver contratos	CRUD	R	R	R	R	R
Crear contrato	CRUD	-	-	-	U	U
Editar contrato	CRUD	U	-	-	U	U
<i>Actividades</i>						
Ver actividades	CRUD	R	R	R	R	R
Crear actividad	CRUD	C	-	-	U	U
Editar actividad	CRUD	U	-	-	U	U
<i>Reportes</i>						
Ver reportes	R	R	R	-	R	CR
Generar reportes	-	-	-	-	-	C
Exportar datos	R	R	R	-	R	CR

*Nota: * significa acceso solo a información propia*

7.2.3. Restricciones Especiales

- **Artist:** Solo puede editar su propio perfil
- **Apprentice:** Solo puede ver su propio perfil y evaluaciones
- **Staff:** No puede crear o eliminar usuarios

- **Manager:** No puede crear agencias ni aprobar contratos principales
- **Director:** Acceso de lectura a todos, escritura en áreas supervisadas
- **Admin:** Acceso irrestricto (con auditoría completa)

7.3. Matriz de Permisos por Rol

7.3.1. AuthMiddleware

Verifica que la solicitud incluya JWT válido:

- Header “Authorization” presente
- Token con formato “Bearer <token>”
- JWT válido y no expirado
- Usuario existe en base de datos

7.3.2. RoleMiddleware

Verifica que el usuario tenga el rol requerido:

- Usuario autenticado
- Rol está en lista de permitidos

7.4. Protección de Datos Sensibles

- Contraseñas: Hasheadas con bcrypt (10 rounds)
- Tokens: Expiración 24 horas (refresh: 7 días)
- HTTPS: Requerido en producción
- CORS: Configurado para dominios específicos
- Rate Limiting: 100 solicitudes por minuto por IP

Capítulo 8

Documentación de APIs

8.1. Base URL

Development: `http://localhost:3000/api`
Production: `https://kpop-api.example.com/api`

8.2. Headers Requeridos

Content-Type: `application/json`
Authorization: `Bearer <JWT_TOKEN>`
X-Request-ID: `<UUID>`

8.3. Códigos HTTP Utilizados

Cuadro 8.1: HTTP Status Codes

Código	Significado	Descripción
200	OK	Solicitud exitosa
201	Created	Recurso creado
204	No Content	Éxito sin contenido
400	Bad Request	Solicitud inválida
401	Unauthorized	No autenticado
403	Forbidden	No autorizado
404	Not Found	Recurso no existe
409	Conflict	Conflicto (ej: email duplicado)
500	Server Error	Error interno

8.4. Endpoints por Módulo

8.4.1. Autenticación

POST `/auth/login` - Login de usuario
POST `/auth/logout` - Logout

POST	/auth/refresh	- Refrescar token
GET	/auth/profile	- Obtener perfil
PUT	/auth/profile	- Actualizar perfil

8.4.2. Usuarios

GET	/users	- Listar usuarios
GET	/users/:id	- Obtener usuario
POST	/users	- Crear usuario
PUT	/users/:id	- Actualizar usuario
DELETE	/users/:id	- Eliminar usuario
GET	/users/search	- Buscar usuarios

8.4.3. Agencias

GET	/agencies	- Listar agencias
GET	/agencies/:id	- Obtener agencia
POST	/agencies	- Crear agencia
PUT	/agencies/:id	- Actualizar agencia
DELETE	/agencies/:id	- Eliminar agencia

8.4.4. Grupos

GET	/groups	- Listar grupos
GET	/groups/:id	- Obtener grupo
POST	/groups	- Crear grupo
PUT	/groups/:id	- Actualizar grupo
DELETE	/groups/:id	- Eliminar grupo
GET	/groups/:id/members	- Miembros del grupo

8.4.5. Aprendices

GET	/apprentices	- Listar aprendices
GET	/apprentices/:id	- Obtener aprendiz
POST	/apprentices	- Crear aprendiz
PUT	/apprentices/:id	- Actualizar aprendiz
DELETE	/apprentices/:id	- Eliminar aprendiz

Capítulo 9

Validaciones y Restricciones

9.1. Validaciones por Entidad

9.1.1. Usuario

Cuadro 9.1: Validaciones de Usuario

Campo	Restricción	Tipo
email	Formato email, único, requerido	Validación
nombre	Texto 2-255 caracteres	Validación
password	Mín. 8 caracteres, mayúsculas, minúsculas, números, símbolos	Validación
rol	ENUM de 6 roles disponibles	Validación

9.1.2. Aprendiz

Cuadro 9.2: Validaciones de Aprendiz

Campo	Restricción	Tipo
nombre	Texto 2-255 caracteres, requerido	Validación
dateOfBirth	Edad 15-50 años, fecha pasada	Validación
agencyId	Foreign Key válido, requerido	Validación

9.2. Restricciones de Negocio

Cuadro 9.3: Restricciones de Negocio

Restricción	Descripción	Validación
Email único	No puede haber dos usuarios con mismo email	BD + API
Edad mínima	Aprendices deben tener mínimo 15 años	API
Edad máxima	Aprendices deben tener máximo 50 años	API
Fecha debut	Debe ser anterior a hoy	API
Contrato válido	endDate debe ser $\geq$ startDate	API
Nombre único	Cada grupo debe tener nombre único	BD

Capítulo 10

Flujos de Negocio

10.1. Flujo 1: Login de Usuario

El proceso de autenticación sigue estos pasos:

1. Usuario ingresa email y contraseña
2. Backend valida credenciales en BD
3. Si son inválidas: Retornar error 401
4. Si son válidas: Generar JWT token (exp: 24 horas)
5. Generar refresh token (exp: 7 días)
6. Retornar tokens y datos de usuario

10.2. Flujo 2: Crear Nuevo Grupo

El proceso de creación de grupo:

1. Usuario (MANAGER/ADMIN) ingresa datos del grupo
2. Frontend valida datos localmente
3. Si hay errores: Mostrar mensaje
4. Si son válidos: POST /groups con datos
5. Backend valida autorización (requiere MANAGER/ADMIN)
6. Si no autorizado: Retornar error 403
7. Backend valida datos
8. Si falla: Retornar error 400
9. Verificar nombre único
10. Si existe: Retornar error 409
11. Si es único: Crear grupo en BD

12. Registrar en auditoría
13. Retornar grupo creado (201)

10.3. Flujo 3: Evaluar Aprendiz

El proceso de evaluación automáticamente determina estado:

1. MANAGER accede a módulo de Evaluación
2. Sistema obtiene lista de aprendices ACTIVE
3. MANAGER selecciona aprendiz e ingresa evaluación
4. Backend valida puntaje (0-100)
5. Si es inválido: Retornar error
6. Si es válido: Guardar evaluación
7. Verificar promedio últimas 5 evaluaciones
8. Si promedio ≥ 80 : Estado = GRADUATED, crear Usuario ARTIST
9. Si promedio < 40 : Estado = SUSPENDED, notificar supervisor
10. Si 40-79: Estado = ACTIVE, mantener
11. Actualizar registro

Capítulo 11

Pruebas

11.1. Estrategia de Testing

11.1.1. Unit Tests

Enfoque en funciones individuales y validaciones:

- Cobertura: >80 %
- Framework: Jest
- Ejemplo: Validar formato de email

11.1.2. Integration Tests

Enfoque en APIs y flujos completos:

- Cobertura: Endpoints críticos
- Framework: Jest + Supertest
- Ejemplo: Test de login con credenciales válidas

11.1.3. E2E Tests

Enfoque en flujos de usuario completos:

- Herramienta: Cypress o Playwright
- Ejemplo: Crear grupo y agregar miembros

11.2. Test Cases Críticos

Cuadro 11.1: Test Cases

Caso	Descripción	Resultado Esperado
TC-AUTH-001	Login con credenciales válidas	Status 200, tokens retornados
TC-AUTH-002	Login con email inválido	Status 401, error de autenticación
TC-USER-001	Crear usuario con email duplicado	Status 409, error de conflicto
TC-USER-002	Crear usuario con datos válidos	Status 201, usuario creado
TC-GROUP-001	Crear grupo sin autenticación	Status 401, no autorizado
TC-GROUP-002	Crear grupo con nombre duplicado	Status 409, conflicto
TC-APPRENTICE-001	Crear aprendiz con edad <15	Status 400, validación fallida
TC-APPRENTICE-002	Crear aprendiz con edad válida	Status 201, aprendiz creado

Capítulo 12

Conclusiones

12.1. Logros Alcanzados

- ✓ Sistema robusto: Arquitectura escalable con separación clara de capas
- ✓ Seguridad: Implementación de JWT, roles y controles de acceso
- ✓ Escalabilidad: Diseño modular que permite crecimiento
- ✓ Mantenibilidad: Código limpio siguiendo Clean Architecture
- ✓ Funcionalidad: 14 módulos cubriendo necesidades de agencias K-Pop

12.2. Beneficios para la Organización

- **Eficiencia:** Automatización de procesos manuales
- **Datos Centralizados:** Una única fuente de verdad
- **Seguridad:** Control granular de acceso y auditoría
- **Reportes:** Visualización de datos en tiempo real
- **Escalabilidad:** Infraestructura preparada para crecimiento

12.3. Recomendaciones Futuras

1. Implementar Caché: Redis para mejorar performance
2. Agregar WebSockets: Para notificaciones en tiempo real
3. Expandir Mobile: Aplicación nativa para iOS/Android
4. Analytics: Dashboard de métricas del sistema
5. Integración: APIs externas (Spotify, Instagram, etc.)

Apéndice A

Anexos

A.1. Datos de Prueba

Usuarios disponibles para testing:

ADMIN:

Email: admin@kpop.com
Password: AdminPass123!

MANAGER:

Email: manager@kpop.com
Password: ManagerPass123!

ARTIST:

Email: artist@kpop.com
Password: ArtistPass123!

A.2. Variables de Entorno Backend

```
NODE_ENV=development
PORT=3000
DATABASE_URL=postgresql://user:password@localhost:5432/kpop_db
JWT_SECRET=your-secret-key-here
REFRESH_TOKEN_SECRET=your-refresh-secret
JWT_EXPIRATION=24h
CORS_ORIGIN=http://localhost:5173
```

A.3. Referencia Rápida de Errores

Cuadro A.1: Errores Comunes

Código	Mensaje	Solución
AUTH_001	Invalid credentials	Verificar email y contraseña
AUTH_002	Token expired	Usar refresh token
VAL_001	Email format invalid	Verificar formato de email
VAL_002	Age out of range	Aprendiz 15-50 años
AUTH_003	Unauthorized	Verificar permisos del rol
DB_001	Duplicate entry	El registro ya existe

Apéndice B

Flujos de Negocio

B.1. Introducción

Los flujos de negocio describen cómo el sistema procesa los casos de uso principales del negocio de gestión de agencias K-pop. Cada flujo incluye validaciones, decisiones y cambios de estado automáticos.

B.2. Flujo 1: Autenticación de Usuario

B.2.1. Descripción

Un usuario intenta acceder al sistema proporcionando credenciales (email y contraseña).

B.2.2. Participantes

- Usuario (Cliente Frontend)
- Sistema (Servidor Backend)
- Base de Datos

B.2.3. Secuencia del Flujo

1. **Inicio:** Usuario navega a `/login` y ve formulario
2. **Acción 1:** Usuario ingresa email y contraseña
3. **Validación 1:** Frontend valida formato de email localmente
 - X Inválido → Mostrar error "Formato de email incorrecto"
 - OK Válido → Continuar
4. **Acción 2:** Usuario hace clic en botón Login
5. **Acción 3:** Frontend envía POST `/auth/login` con credenciales
6. **Validación 2:** Backend valida estructura de datos

X Campos vacíos → Error 400 Campos requeridos"

OK Válido → Continuar

7. **Validación 3:** Backend busca usuario en BD por email

X No encontrado → Error 401 .Email o contraseña incorrectos"

OK Encontrado → Continuar

8. **Validación 4:** Backend valida contraseña (bcrypt compare)

X No coincide → Error 401 .Email o contraseña incorrectos"

OK Coincide → Continuar

9. **Validación 5:** Backend verifica estado de usuario

X Inactivo/Suspendido → Error 403 Cuenta desactivada"

OK Activo → Continuar

10. **Acción 4:** Backend genera JWT (access token) válido por 24 horas

11. **Acción 5:** Backend genera refresh token válido por 7 días

12. **Acción 6:** Backend registra login en auditoría con timestamp e IP

13. **Retorno:** Backend retorna tokens al frontend (200 OK)

14. **Acción 7:** Frontend guarda tokens en localStorage/sessionStorage

15. **Acción 8:** Frontend redirige a dashboard según rol

16. **Fin:** Usuario autenticado en el sistema

2. **Acción 1:** Manager busca/filtra aprendiz específico
3. **Acción 2:** Manager hace clic en botón ".Evaluar"
4. **Validación 1:** Sistema verifica permisos de manager
 - X Sin permisos → Error 403
 - OK Con permisos → Continuar
5. **Acción 3:** Se abre formulario de evaluación
6. **Acción 4:** Manager ingresa puntaje (0-100)
7. **Validación 2:** Sistema valida puntaje
 - X Fuera de rango → Error "Puntaje debe estar entre 0 y 100"
 - OK Válido → Continuar
8. **Acción 5:** Manager opcionalmente agrega comentarios
9. **Acción 6:** Manager hace clic en "Guardar Evaluación"
10. **Acción 7:** Backend guarda evaluación en BD con timestamp
11. **Acción 8:** Backend recupera últimas 5 evaluaciones del aprendiz
12. **Acción 9:** Backend calcula promedio automáticamente
13. **Decisión 1:** ¿Promedio ≥ 80 ?
 - **SÍ** → Ir a Ramificación A (Graduación)
 - **NO** → Ir a Decisión 2
14. **Decisión 2:** ¿Promedio < 40 ?
 - **SÍ** → Ir a Ramificación B (Suspensión)
 - **NO** → Ir a Acción 12

Ramificación A: Graduación (Promedio ≥ 80)

1. Cambiar estado de aprendiz a GRADUADO
2. Crear nuevo Usuario en BD con:
 - email = apellido + número aleatorio@kpop.local
 - rol = ARTIST
 - nombre = nombre del aprendiz
 - agencyId = misma agencia del aprendiz
3. Generar contraseña temporal aleatoria
4. Enviar email al aprendiz con credenciales y enlace para cambiar contraseña
5. Registrar evento en auditoría: ".APPRENTICE_GRADUATED"
6. Notificar a director/admin de la graduación

7. Retornar confirmación al manager

Ramificación B: Suspensión (Promedio < 40)

1. Cambiar estado de aprendiz a SUSPENDIDO
2. Generar reporte de bajo desempeño
3. Enviar notificación a supervisor asignado
4. Registrar evento en auditoría: .APPRENTICE_SUSPENDED"
5. Crear tarea para revisión: Revisar plan de recuperación para [Aprendiz]"
6. Retornar confirmación al manager

Continuación Normal (Promedio 40-79)

1. Mantener estado como ACTIVO
2. Registrar evento en auditoría: .APPRENTICE_EVALUATED"
3. Retornar confirmación al manager
4. **Fin:** Evaluación guardada y procesada

B.4. Flujo 3: Crear Nuevo Grupo

B.4.1. Descripción

Un manager crea un nuevo grupo musical bajo la agencia, definiendo nombre, concepto y otros detalles.

B.4.2. Validaciones Críticas

- Nombre del grupo debe ser único en todo el sistema
- Número de miembros debe estar entre 1 y 20
- Fecha de debut no puede ser futura
- Manager debe tener permisos (MANAGER/ADMIN/DIRECTOR)
- Grupo debe pertenecer a una agencia válida

B.4.3. Resultado

Nuevo registro de grupo creado con estado INACTIVO hasta que tenga al menos un artista.

B.5. Flujo 4: Registrar Ingreso

B.5.1. Descripción

Se registra un ingreso (streaming, concierto, merchandising) que genera dinero para un artista o grupo. El sistema distribuye automáticamente según contratos.

B.5.2. Cálculo de Distribución

1. Buscar todos los contratos activos del grupo/artista
2. Para cada contrato activo, aplicar porcentaje de distribución
3. Registrar transacción de distribución en auditoría
4. Actualizar saldos pendientes de pago
5. Generar notificación de pago pendiente si aplica

B.6. Restricciones de Transición de Estado

No todos los cambios de estado son permitidos. Las transiciones válidas se muestran en la siguiente tabla:

Cuadro B.1: Transiciones de Estado de Aprendiz Permitidas

Estado Actual	Puede Cambiar a	Desencadenante
ACTIVO	GRADUADO	Promedio evaluación ≥ 80
ACTIVO	SUSPENDIDO	Promedio evaluación < 40
ACTIVO	INACTIVO	Acción manual de admin
SUSPENDIDO	ACTIVO	Reactivación manual por admin
GRADUADO	—	Estado permanente (no vuelve)

Apéndice C

Guía de Mantenimiento

C.1. Introducción

Esta sección proporciona información esencial para desarrolladores y administradores de sistema que realicen mantenimiento, depuración y actualización del sistema K-POP World.

C.2. Estructura de Directorios

```
backend/  
  src/  
    application/      # Logica de aplicacion  
    domain/           # Entidades e interfaces  
    infrastructure/   # Configuracion y servicios  
    presentation/     # Controladores y rutas  
    plugins/          # Modulos externos  
    generated/        #Codigo generado (Prisma)  
    tests/            # Suite de pruebas  
  prisma/  
    schema.prisma     # Esquema de BD  
    migrations/       # Historial migraciones  
  tsconfig.json       # Config TypeScript  
  jest.config.ts      # Config pruebas  
  package.json        # Dependencias  
  nodemon.json        # Config reloading  
  
frontend/  
  src/  
    components/       # Componentes React  
    pages/            # Paginas/vistas  
    contexts/         # Context API  
    types/            # Tipos TypeScript  
    config/           # Configuracion  
  public/             # Archivos estaticos
```

```
vite.config.ts      # Config Vite
package.json        # Dependencias
```

C.3. Procedimientos de Diagnóstico

C.3.1. Verificación de Base de Datos

1. Conectar a PostgreSQL:

```
psql -U kpop_user -d kpop_db -h localhost
```

2. Verificar estado de tablas:

```
SELECT table_name FROM information_schema.tables
WHERE table_schema = 'public';
```

3. Revisar integridad referencial:

```
SELECT constraint_name, table_name, violation_count
FROM pg_constraint WHERE table_name LIKE '%';
```

4. Limpiar caché de conexiones abandonadas:

```
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE username = 'kpop_user' AND state = 'idle';
```

C.3.2. Verificación de Servidor

1. Revisar logs del servidor:

```
tail -f logs/server.log
grep -i "error" logs/server.log
```

2. Probar conectividad de API:

```
curl -X GET http://localhost:3000/health
curl -X GET http://localhost:3000/api/users
-H "Authorization: Bearer TOKEN"
```

3. Verificar puertos en uso:

```
netstat -an | grep 3000
netstat -an | grep 5432
```

4. Revisar variables de entorno:

```
cat .env | grep -v PRIVATE
```

C.4. Procedimientos de Depuración

C.4.1. Depuración Node.js

```
# Modo debug
node --inspect=0.0.0.0:9229 src/index.ts
```

```
# Desde Chrome: chrome://inspect
# Desde VS Code: agregar configuración launch.json
{
  "type": "node",
  "request": "attach",
  "name": "Attach",
  "port": 9229
}
```

C.4.2. Depuración TypeScript/Prisma

1. Generar cliente Prisma actualizado:

```
npx prisma generate
```

2. Visualizar esquema:

```
npx prisma studio
```

3. Validar migraciones:

```
npx prisma migrate status
npx prisma migrate resolve --applied <migration_name>
```

4. Limpiar base de datos (CUIDADO):

```
npx prisma migrate reset
npx prisma db seed
```

C.5. Procedimientos de Backup y Restauración

C.5.1. Backup de Base de Datos

```
# Backup completo
pg_dump -U kpop_user -d kpop_db -F c -f backup.dump

# Backup solo estructura
pg_dump -U kpop_user -d kpop_db -s > schema.sql

# Backup con compresión
pg_dump -U kpop_user -d kpop_db | gzip > backup_$(date +%Y%m%d).sql.gz
```

C.5.2. Restauración de Base de Datos

```
# Desde archivo dump
pg_restore -U kpop_user -d kpop_db backup.dump

# Desde SQL comprimido
gunzip -c backup_20260116.sql.gz | psql -U kpop_user -d kpop_db

# Restaurar tabla específica
```

```
pg_restore -U kpop_user -d kpop_db -t nombre_tabla backup.dump
```

C.6. Problemas Comunes y Soluciones

Cuadro C.1: Problemas de Mantenimiento

Problema	Causa Probable	Solución
ECONNREFUSED (5432)	PostgreSQL no está corriendo	Iniciar: <code>sudo systemctl start postgresql</code>
JWT malformed	Token inválido o expirado	Generar nuevo token, verificar SECRET
Prisma Error: Cannot find	Cliente no generado	<code>npx prisma generate</code>
Módulo no encontrado	Dependencia no instalada	<code>npm install</code> , verificar package.json
Query timeout	DB lenta o cuelgue	Optimizar índices, revisar logs
CORS error	Origen no permitido	Actualizar CORS_ORIGIN en .env

C.7. Monitoreo y Performance

C.7.1. Índices de Base de Datos

```
-- Listar índices existentes
SELECT indexname, indexdef FROM pg_indexes
WHERE tablename = 'usuario';

-- Crear índice en campo frecuentemente consultado
CREATE INDEX idx_usuario_email ON usuario(email);

-- Analizar query performance
EXPLAIN ANALYZE SELECT * FROM usuario
WHERE email = 'user@example.com';
```

C.7.2. Métricas de Servidor

1. CPU y memoria:

```
ps aux | grep node
top -p $(pgrep -f "node src")
```

2. Conexiones activas:

```
psql -c "SELECT count(*) FROM pg_stat_activity"
```

3. Tamaño de base de datos:

```
psql -c "SELECT pg_size_pretty(pg_database_size('kpop_db'))"
```

C.8. Consideraciones de Seguridad

C.8.1. Gestión de Secretos

- **Nunca** commit `.env` a repositorio
- Utilizar secretos diferentes para cada ambiente (dev/prod)
- Rotar tokens JWT periódicamente (cada 30-90 días)
- Cambiar contraseñas de BD regularmente
- Utilizar variables de entorno para credentials sensibles

C.8.2. Auditoría y Logs

- Todos los cambios de datos se registran en base de datos
- Logs incluyen: usuario, acción, timestamp, valores antiguos/nuevos
- Revisar logs regularmente para detectar anomalías
- Mantener logs durante mínimo 3 meses
- Implementar alertas para operaciones críticas

C.8.3. Actualizaciones y Parches

- Revisar vulnerabilidades: `npm audit`
- Actualizar dependencias: `npm update`
- Probar cambios en ambiente de staging antes de producción
- Mantener historial de cambios y rollback plan

Apéndice D

Diagrama de Clases UML

D.1. Introducción

Esta sección describe la estructura de clases del sistema según el patrón Clean Architecture. El diagrama visual se puede encontrar en `backend/src/diagram.drawio`.

D.2. Capas de la Arquitectura

D.2.1. Domain Layer (Capa de Dominio)

Contiene las entidades de negocio y reglas de dominio:

- **Usuario:** Entidad principal de acceso
 - Propiedades: id, email, nombre, password, rol, agencyId
 - Métodos: validarCredenciales(), obtenerPermisos()
- **Agencia:** Entidad empresarial
 - Propiedades: id, nombre, dirección, fechaFundación, estado
 - Métodos: agregarGrupo(), obtenerGrupos(), calcularIngresos()
- **Grupo:** Entidad musical
 - Propiedades: id, nombre, debut, status, miembros, agencyId
 - Métodos: agregarMiembro(), removerMiembro(), obtenerAlbumes()
- **Artista:** Integrante de grupo
 - Propiedades: id, nombreArtístico, début, status, groupId
 - Métodos: cambiarGrupo(), obtenerPremios(), actualizarEstado()
- **Aprendiz:** Nuevo talento en formación
 - Propiedades: id, nombre, fechaNacimiento, agencyId, estado
 - Métodos: promoverArtista(), evaluarProgreso()

- **Álbum:** Producción discográfica
 - Propiedades: id, título, groupId, fechaLanzamiento, numCanciones, copiaVendidas
 - Métodos: agregarCanción(), calcularDuración(), obtenerEstadísticas()
- **Canción:** Composición musical
 - Propiedades: id, título, albumId, duración, reproducciones
 - Métodos: incrementarReproducciones(), obtenerMetadata()
- **Contrato:** Acuerdo vinculante
 - Propiedades: id, tipo, parte1Id, parte2Id, fechaInicio, fechaFin, condiciones
 - Métodos: esVigente(), calcularVigencia(), renovar()
- **Premio:** Reconocimiento recibido
 - Propiedades: id, nombre, groupId/artistaId, fecha, categoría
 - Métodos: obtenerDetalles(), verificarAutenticidad()
- **Actividad:** Evento o presentación
 - Propiedades: id, tipo, fecha, lugar, participantes, estado
 - Métodos: agregarParticipante(), cambiarFecha(), finalizarActividad()
- **Ingreso:** Registro económico
 - Propiedades: id, tipo, monto, fecha, actividadId, distribución
 - Métodos: calcularDistribución(), obtenerReporte()

D.2.2. Application Layer (Capa de Aplicación)

Contiene la lógica de casos de uso:

- **UseCase (Interfaz Base):** Define el contrato
 - Método abstracto: execute(input): Output
- Casos de uso específicos (ejemplos):
 - CrearAgenciaUseCase
 - AgregarGrupoUseCase
 - RegistrarActividadUseCase
 - CalcularIngresosUseCase
 - GenerarReporteUseCase
- **DTOs (Data Transfer Objects):**
 - CreateAgencyDTO
 - UpdateGroupDTO

- RegisterActivityDTO
- GenerateReportDTO

D.2.3. Infrastructure Layer (Capa de Infraestructura)

Implementa detalles técnicos:

- **Prisma Client:** Cliente ORM automático
 - Generado automáticamente desde schema.prisma
 - Proporciona tipado fuerte
 - Métodos: create(), findUnique(), update(), delete()
- **Repositories:**
 - UserRepository
 - AgencyRepository
 - GroupRepository
 - ActivityRepository
 - IncomeRepository
- **Database Connection:**
 - PostgreSQL Connection Pool
 - Migraciones Prisma
 - Seeds de datos iniciales
- **Servicios Externos:**
 - JWT Handler (autenticación)
 - Email Service (notificaciones)
 - Logger Service (auditoría)

D.2.4. Presentation Layer (Capa de Presentación)

Maneja las solicitudes HTTP:

- **Controladores:**
 - AuthController
 - UserController
 - AgencyController
 - GroupController
 - ArtistController
 - ApprenticeController

- ActivityController
- IncomeController
- **Rutas:**
 - POST /auth/login
 - GET/POST /api/agencies
 - GET/POST /api/groups
 - GET/POST /api/activities
 - GET /api/reports
- **Middleware:**
 - AuthenticationMiddleware
 - AuthorizationMiddleware
 - ValidationMiddleware
 - ErrorHandlingMiddleware
 - LoggingMiddleware

D.3. Patrones de Diseño

D.3.1. Dependency Injection

```
// Contenedor de IoC
class Container {
  private services: Map<string, any>

  register(name: string, factory: Function)
  resolve<T>(name: string): T
}

// Uso en controladores
class AgencyController {
  constructor(private agencyUseCase: AgencyUseCase) {}

  async createAgency(req: Request, res: Response) {
    const result = await this.agencyUseCase.execute(req.body)
    res.json(result)
  }
}
```

D.3.2. Repository Pattern

```
interface IRepository<T> {
  create(data: Partial<T>): Promise<T>
}
```

```

    findAll(): Promise<T[]>
    findById(id: number): Promise<T | null>
    update(id: number, data: Partial<T>): Promise<T>
    delete(id: number): Promise<boolean>
  }

class UserRepository implements IRepository<User> {
  constructor(private prisma: PrismaClient) {}

  async findById(id: number): Promise<User | null> {
    return this.prisma.usuario.findUnique({
      where: { id }
    })
  }
}

```

D.4. Relaciones entre Clases

Cuadro D.1: Relaciones UML

Clase A	Relación	Clase B
Agencia	Composición (1:N)	Grupo
Grupo	Composición (1:N)	Artista
Grupo	Composición (1:N)	Álbum
Álbum	Composición (1:N)	Canción
Contrato	Asociación (M:N)	Usuario/Artista
Actividad	Agregación	Participantes
Ingreso	Asociación	Actividad
UseCase	Dependencia	Repository
Controller	Dependencia	UseCase